

Short Write-Ups

1. In your own words, what is the difference between a delegate and a lambda expression?

A delegate is a type — it defines the signature that any method must match in order to be stored in a variable of that delegate type (for example, `public delegate int MathOperation(int a, int b);`). It's essentially a template describing the shape of method that's allowed. A lambda expression, on the other hand, is not a type at all; it's a concise way of writing the actual method body — the logic itself — that gets stored inside a variable of that delegate type, for example `(a, b) => a + b`. In short: the delegate is the container (the type), and the lambda expression is a quick way to fill that container with real, executable logic, instead of writing out a full, separately named method and referencing it.

2. Why is `var` still statically typed even though the type isn't written explicitly?

Because the compiler determines the actual, concrete type at compile time, not at runtime. When you write `var x = 5;`, the compiler looks at the right-hand side (5) and decides that `x` is of type `int` at that exact moment of compilation — after that, `x` behaves exactly like any ordinary `int` variable. If you tried to assign a string to it afterward, you'd get a compile-time error, exactly as you would with an explicitly typed variable. This is very different from dynamically typed languages (or C#'s own `dynamic` keyword), where the type is only resolved at runtime and the compiler can't verify it ahead of time. `var` is purely syntactic sugar that saves you from writing the type yourself — the type itself is still fixed, known, and enforced just as strictly as with any explicitly declared type.

3. Give one real scenario where an anonymous type is more convenient than creating a full class.

A very common example is building a small, temporary shape of data — often inside a LINQ query or a quick loop — that's only needed within that one method and never reused anywhere else in the project. For instance, given a list of `Employee` objects, if you just want to print each employee's name alongside their salary after tax, without needing that combined shape anywhere else in the program: `var summary = new { Name = emp.Name, NetSalary = emp.Salary * 0.9 };`. Creating a full, separately named class just to hold this one-off, throwaway value would be overkill — you'd need to pick a name, decide on access modifiers, write a constructor, all for something that's used once and then discarded. An anonymous type gives you exactly what you need (clearly named properties) with none of that extra overhead, which makes it especially convenient for temporary query results or single-use data groupings like this.